

Taller de proyectos 2 – ingeniería de Sistemas e informática

CONSIGNA DEL PROYECTO DE FIN DE CURSO

PLATAFORMA DE GESTIÓN DE PROYECTOS COLABORATIVOS CON INTEGRACIÓN DE INTELIGENCIA ARTIFICIAL

Objetivo

Desarrollar una aplicación web innovadora utilizando el stack MERN (MongoDB, Express.js, React.js, Node.js) que permita la gestión colaborativa de proyectos, integrando funcionalidades impulsadas por Inteligencia Artificial (IA) para mejorar la productividad y experiencia del usuario. La aplicación debe ser desarrollada siguiendo buenas prácticas de ingeniería de software, incluyendo control de versiones, pruebas automatizadas, contenerización y despliegue en entornos Dockerizados.

Requisitos Generales

1. Stack Tecnológico
 - Frontend: React.js con componentes reutilizables y diseño responsive.
 - Backend: Node.js con Express.js para manejar las APIs RESTful o GraphQL.
 - Base de Datos: MongoDB para almacenamiento de datos estructurados.
 - Integración de IA: Identificar y desarrollar al menos dos funcionalidades clave donde se utilice IA (ver sugerencias más abajo).
2. Control de Versiones
 - Utilizar Git como sistema de control de versiones.
 - Publicar el código fuente en un repositorio público en GitHub .
 - Implementar un flujo de trabajo claro para el control de versiones (por ejemplo, Git Flow o GitHub Flow), asegurando ramas separadas para desarrollo (**develop**), características (**feature/***) y producción (**main**).
3. Pruebas
 - Incluir pruebas unitarias y de integración para el backend.
 - Realizar pruebas de interfaz de usuario (E2E) para el frontend utilizando herramientas como Jest , Cypress o React Testing Library .
 - Documentar los casos de prueba y resultados esperados.
4. Contenerización
 - Desarrollar la aplicación utilizando Docker para garantizar consistencia entre los entornos de desarrollo y producción.
 - Crear un archivo **docker-compose.yml** que configure los contenedores necesarios para el frontend, backend, base de datos y cualquier servicio adicional (por ejemplo, servicios de IA).
 - Asegurar que la aplicación pueda desplegarse fácilmente en un entorno de producción utilizando Docker.
5. Metodología Ágil
 - Seguir una metodología ágil (Scrum o Kanban) durante el desarrollo del proyecto.
 - Dividir el trabajo en sprints semanales con entregas incrementales.
 - Documentar cada etapa del desarrollo, incluyendo backlog, historias de usuario y retrospectivas.
6. Documentación
 - Incluir un archivo **README.md** detallado con instrucciones claras para configurar y ejecutar la aplicación.
 - Documentar el diseño arquitectónico, diagramas de flujo y especificaciones técnicas.
 - Explicar cómo se integra la IA en las funcionalidades seleccionadas.

Taller de proyectos 2 – ingeniería de Sistemas e informática

Funcionalidades Principales

La plataforma debe permitir la gestión colaborativa de proyectos con las siguientes características

1. Gestión de Proyectos
 - Creación, edición y eliminación de proyectos.
 - Asignación de tareas a miembros del equipo con fechas límite y prioridad.
 - Visualización de tableros Kanban para organizar tareas.
2. Comunicación en Tiempo Real
 - Chat integrado para facilitar la comunicación entre los miembros del equipo.
 - Notificaciones push para actualizaciones importantes.
3. Integración de IA (Funcionalidades Propuestas): Los estudiantes deben identificar y desarrollar al menos dos funcionalidades donde se utilice IA. Algunas sugerencias incluyen
 - Asistente Virtual: Un chatbot basado en IA que ayude a los usuarios a gestionar sus tareas, responder preguntas comunes o sugerir próximos pasos.
 - Tecnologías sugeridas: OpenAI API , Dialogflow o modelos personalizados.
 - Predicción de Tiempos: Usar IA para predecir la duración estimada de las tareas basándose en datos históricos.
 - Tecnologías sugeridas: TensorFlow.js , scikit-learn o AWS SageMaker .
 - Automatización de Tareas: Clasificación automática de tareas según su urgencia o categoría utilizando NLP (Procesamiento del Lenguaje Natural).
 - Tecnologías sugeridas: Hugging Face Transformers , spaCy .
4. Panel de Administración
 - Dashboard con estadísticas visuales sobre el progreso del proyecto (gráficos, métricas de rendimiento).
 - Rol de administrador para gestionar usuarios y permisos.

Entregables

1. Código Fuente
 - Repositorio público en GitHub con commits organizados y mensajes claros.
 - Archivos Docker y **docker-compose.yml** completos.
2. Documentación
 - Archivo **README.md** con instrucciones de instalación, uso y despliegue.
 - Diagramas de arquitectura y flujo de datos.
3. Demostración
 - Una presentación (video o en vivo) que muestre el funcionamiento de la aplicación y explique cómo se integró la IA.
4. Informe Final
 - Breve informe técnico que detalle las decisiones de diseño, tecnologías utilizadas, desafíos enfrentados y soluciones implementadas.

Criterios de Evaluación

1. Funcionalidad Completa: La aplicación cumple con todas las funcionalidades requeridas y funciona correctamente.
2. Uso de IA: Las funcionalidades de IA están bien integradas y aportan valor significativo al proyecto.
3. Calidad del Código: El código es limpio, modular y sigue buenas prácticas de programación.
4. Control de Versiones: El repositorio está bien organizado, con un flujo de trabajo claro y commits significativos.
5. Despliegue Dockerizado: La aplicación puede desplegarse fácilmente en un entorno Dockerizado.
6. Documentación: La documentación es clara, completa y útil para futuros desarrolladores.

Taller de proyectos 2 – ingeniería de Sistemas e informática

Rúbrica de Evaluación

CRITERIO	INDICADORES	SOBRESALIENTE (3)	SUFICIENTE (2)	EN DESARROLLO (1)	INSATISFACTORIO (0)
Funcionalidad Completa	- Cumplimiento de todas las funcionalidades requeridas. - Corrección y usabilidad de la aplicación.	Todas las funcionalidades están implementadas correctamente, funcionan sin errores y son intuitivas para el usuario.	La mayoría de las funcionalidades están implementadas, pero algunas tienen errores menores o limitaciones.	Algunas funcionalidades clave no están implementadas o presentan errores significativos.	Las funcionalidades básicas no están implementadas o la aplicación no es funcional.
Calidad del Código	- Estructura modular y organización del código. - Uso de buenas prácticas de programación.	El código está bien organizado, sigue buenas prácticas (DRY, KISS), es legible y cuenta con comentarios claros.	El código está organizado, pero presenta algunos problemas de estructura o falta de comentarios.	El código es confuso, repetitivo o difícil de entender, con pocos comentarios o sin seguir buenas prácticas.	El código es caótico, plagado de errores y no sigue ninguna estructura clara.
Control de Versiones	- Uso de Git y GitHub. - Flujo de trabajo claro y commits significativos.	El repositorio está bien organizado, con un flujo de trabajo claro (por ejemplo, Git Flow) y commits descriptivos.	El repositorio está organizado, pero algunos commits carecen de claridad o el flujo de trabajo no es consistente.	El repositorio tiene un flujo de trabajo inconsistente y los commits no son descriptivos ni útiles.	No hay evidencia de uso adecuado de Git o el repositorio no está disponible.
Integración de IA	- Identificación de al menos dos funcionalidades impulsadas por IA. - Calidad de la integración.	Las funcionalidades de IA están bien integradas, aportan valor significativo y funcionan correctamente.	Las funcionalidades de IA están implementadas, pero tienen limitaciones o no aportan un valor claro.	Solo una funcionalidad de IA está implementada o presenta errores significativos.	No se integran funcionalidades de IA o las implementadas no funcionan.
Documentación	- Archivo README.md.	La documentación es completa, detallada y fácil de entender,	La documentación está presente, pero carece de detalles	La documentación es incompleta, confusa o	No hay documentación o es completamente incomprensible.

Taller de proyectos 2 – ingeniería de Sistemas e informática

CRITERIO	INDICADORES	SOBRESALIENTE (3)	SUFICIENTE (2)	EN DESARROLLO (1)	INSATISFACTORIO (0)
	- Diagramas y especificaciones técnicas.	incluyendo instrucciones claras y diagramas precisos.	importantes o diagramas necesarios.	insuficiente para comprender el proyecto.	
Despliegue Dockerizado	- Archivos Docker y docker-compose.yml. - Consistencia entre desarrollo y producción.	La aplicación se despliega correctamente en entornos Dockerizados, con archivos completos y bien configurados.	La aplicación se despliega en Docker, pero presenta pequeños problemas de configuración o inconsistencias.	La aplicación no se despliega correctamente en Docker o los archivos están incompletos.	No hay evidencia de contenerización o el despliegue falla completamente.